APSC 143: Introduction to Computer Programming for Engineers

Assignment #2 - Control and Iteration

Fall 2026

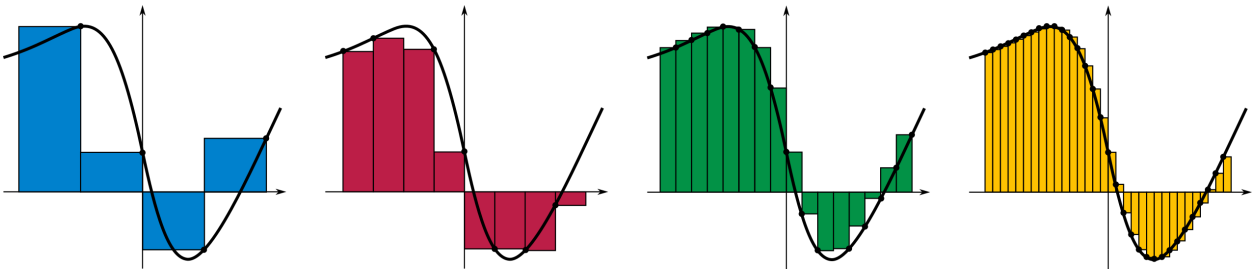


Figure 1: An illustration of the Riemann sum of a function using increasingly narrow rectangles. Original image source: Wikipedia

1 Introduction

In this assignment, you will have to write functions in C that use conditional and iterative statements. These statements will be combined together to calculate complex mathematical expressions.

Specifically, you will calculate the expression, e^{x^2}, and its definite integral, $\int_a^b e^{x^2}$, even though the expression, e^{x^2}, is famously non-integrable.

To achieve this, you will implement at least four different functions that calculate:

1. The value of the inverse of the factorial, $\frac{1}{n!}$, for a given n using loops (or recursion, which will be covered later in the course).
2. The value of the Taylor series expansion of e^{x^2} for a given x using if statements, loops, your function to calculate the inverse of the factorial, and the standard C math functions provided in `math.h`.
3. The value of the analytic integral of the Taylor series expansion of $\int_a^b e^{x^2}$ using if statements, loops, your inverse factorial function, and standard C math functions.
4. The value of the Riemann sum approximation of $\int_a^b e^{x^2}$ using loops and standard C math functions.

The specific programming instructions for the assignment are provided in Section 2. The specific mathematical details on factorials, the Taylor series expansion, and the Riemann sum are in Section 3. Details on how to submit your work are provided in Section 4

 ©2026 Queen's University

2 Instructions

In this assignment, you will create C functions to evaluate the expressions e^{x^2} and $\int_a^b e^{x^2} dx$ in different ways and compare your results. Specifically, you must implement the following four functions:

1. `double inverseFactorial(int n)`
   Calculates and returns $\frac{1}{n!}$ for the given n, i.e., equation (1).
2. `double taylorSeries(double x, double minTermSize)`
   Calculates and returns the approximation of e^{x^2} for the given x by using all terms of the Taylor series expansion whose absolute value is greater than or equal to `minTermSize`, i.e., equation (2). This function will need to use `inverseFactorial`.
3. `double integralTaylorSeries(double lowerLimit, double upperLimit, double minTermSize)`
   Calculates and returns the approximation of $\int_a^b e^{x^2} dx$ between the given `lowerLimit` and `upperLimit` integral limits by using the analytical integration of a Taylor series expansion of e^{x^2} with all terms whose absolute value is greater than or equal to `minTermSize`, i.e., equation (3). This function will need to use `inverseFactorial`.
4. `double riemannSum(double lowerLimit, double upperLimit, int numRectangles)`
   Calculates and returns the approximation of $\int_a^b e^{x^2} dx$ between the given `lowerLimit` and `upperLimit` integral limits by using a Riemann sum with `numRectangles` rectangles, i.e., equation (4).

You will be assessed on the correctness of the output of these four functions. You may choose to implement their internal details however you wish, but **they must have the function signatures (e.g., names, arguments, return values) listed above**.

To receive full marks you should use additional functions to appropriately break up the problem, organize your solution, and avoid duplicating code.

You will likely find it helpful to also write a `main` function that you can use to test your functions against the example output provided in Table 1. We will not assess your `main` function.

Comments are mandatory for this assignment. Add comments as necessary for key pieces in your code to explain what the program is doing, including any function definitions, assumptions, variable declaration, conditional statements, looping conditions, etc.

The results of your functions must match the sample output (Table 1) when rounded to four decimal places; otherwise, the auto grading software will not be able to grade your assignment, which may affect your mark.

2.1 Assumptions

You can make the following assumptions when writing your functions:

 ©2026 Queen's University

1. `double inverseFactorial(int n)`
   The given integer will always be greater than zero, `n` ≥ 0.
2. `double taylorSeries(double x, double minTermSize)`
   The given minimum size of the terms will always be strictly greater than zero, `minTermSize` > 0.
3. `double integralTaylorSeries(double lowerLimit, double upperLimit, double minTermSize)`
   The given bounds will be valid but may be of zero width, `lowerLimit` $\leq$ `upperLimit`. The given minimum size of the terms will always be strictly greater than zero, `minTermSize` > 0.
4. `double riemannSum(double lowerLimit, double upperLimit, int numRectangles)`
   The given bounds will be valid but may be of zero width, `lowerLimit` $\leq$ `upperLimit`. The given number of rectangles will always be greater than or equal to zero, `numRectangles` ≥ 0.

2.2 Existing Functions

Like most programming languages, there are a number of existing, useful functions you can use in C. These functions are no different than the ones you will write yourself, other than that they have been rigorously tested and made publicly available to everyone!

Functions provided by `math.h` that you will need for this assignment include:

- `double fabs(double x)`
  Calculates and returns the absolute value, $|x|$, of a given x.
- `double exp(double x)`
  Calculates and returns e^x for the given x, where $e = 2.7182818\ldots$ is the base of the natural logarithm (i.e., Euler's number).
- `double pow(double x, double y)`
  Calculates and returns x^y for the given x and y.

You can read more about these functions and `math.h` at https://cplusplus.com/reference/cmath/.

3 Mathematical Background

The integral of a function, $\int f(x)dx$, is often described as the area under the curve of the function. The definite integral, $\int_a^b f(x)dx$, is then the area under the curve in the interval $a \leq x \leq b$ and is analytically defined as the difference between the indefinite integral evaluated at the upper and lower limits. Unfortunately, this definition does not work for non-integrable functions, like e^{x^2}. We are going to investigate two methods to calculate the definite integral in these situations, which are:

 ©2026 Queen's University

1. Expressing the function as an infinite sum of terms and then taking the analytical integration of this series. This will yield an expression of the definite integral that is an infinite sum, but that we can approximate from a finite number of terms.
2. Representing the integral as the summation of a series of rectangles. This will yield a different expression of the definite integral in terms of a sum that goes to infinity in the limit, but that we can approximate from a finite number of terms..

3.1 Factorial

To calculate some of these series, we will need to calculate the factorial of a number. The factorial of a nonnegative integer is defined as the product of multiplying the integers between 1 and the number,

$$n! = n \times (n-1) \times (n-2) \times \ldots \times 2 \times 1.$$

It can also be defined *recursively* in terms of the factorial of the next lower integer,

$$n! = n \times (n-1)!$$

Note that the factorial of 0 is defined to be equal to 1,

$$0! = 1,$$

and that the factorial is not defined for negative numbers.

Factorials get very big very quickly and in practice will be too big for many computers at relatively small input numbers (e.g., 20! would likely not be calculated correctly on your computer). This problem can be avoided when the factorial is in the denominator of a fraction by directly calculating the *inverse* factorial,

$$\begin{aligned} g(n) = \frac{1}{n!} &= \frac{1}{n} \times \frac{1}{n-1} \times \frac{1}{n-2} \times \ldots \times \frac{1}{2} \times \frac{1}{1} \\ &= \frac{1}{n} \times \frac{1}{(n-1)!} \\ &= \frac{1}{n} \times g(n-1), \end{aligned} \tag{1}$$

which instead gets very close to zero very quickly.

You will need equations (1) for this assignment and you will need to use it when implementing other functions.

3.2 Taylor Series Expansion

Many functions can be represented as an infinite sum of a series of terms. Perhaps the most famous series expansion is the Taylor series expansion which represents a function as the summation of its derivatives at a specific point. You will learn more about the Taylor series

 ©2026 Queen's University

in APSC 172, but the general expression for a Taylor series expansion of a function, $f(x)$, differentiable at $x = a$ is

$$\begin{aligned} f(x) &= f(a) + \frac{(x-a)}{1!}\frac{d}{dx}f(x)\bigg|_{x=a} + \frac{(x-a)^2}{2!}\frac{d^2}{dx^2}f(x)\bigg|_{x=a} + \frac{(x-a)^3}{3!}\frac{d^3}{dx^3}f(x)\bigg|_{x=a} + \dots \\ &= \sum_{n=0}^{\infty} \frac{(x-a)^n}{n!}\frac{d^n}{dx^n}f(x)\bigg|_{x=a}, \end{aligned}$$

where $\frac{d^n}{dx^n}f(x)\Big|_{x=a}$ is the n-th derivative of $f(x)$ evaluated at $x = a$ and $n!$ is the factorial of the nonnegative integer n.

In this assignment, we will use the Taylor series expansion at $a = 0$, this simpler case is also known as the Maclaurin series,

$$f(x) = \sum_{n=0}^{\infty} \frac{x^n}{n!}\frac{d^n}{dx^n}f(x)\bigg|_{x=0}.$$

The Taylor series expansions of many common functions are known, e.g., $e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!}$, and the Taylor series expansion of other functions can be found by algebraic manipulation of the series expression. Specifically, the Taylor series expansion of e^{x^2} can be shown to be

$$e^{x^2} = \sum_{n=0}^{\infty} \frac{(x^2)^n}{n!} = \sum_{n=0}^{\infty} \frac{1}{n!}x^{2n}. \tag{2}$$

Note that the Taylor series expansion is the sum of an infinite series of terms. If we sum only a *finite* number of terms from the infinite series, then this will be an approximation of the function. This approximation will become more and more accurate as we include more and more terms from the series.

You will need equation (2) for this assignment.

3.3 Analytic Integral of the Taylor Series Expansion

The integral of a function can be calculated as the integral of its Taylor series expansion,

$$\begin{aligned} \int f(x)dx &= \int \sum_{n=0}^{\infty} \frac{x^n}{n!}\frac{d^n}{dx^n}f(x)\bigg|_{x=0} dx \\ &= \sum_{n=0}^{\infty} \frac{1}{n!}\int x^n \frac{d^n}{dx^n}f(x)\bigg|_{x=0} dx. \end{aligned}$$

More specifically, the Taylor series expansion of e^{x^2} in equation (2) lets us write an analytical

 ©2026 Queen's University

expression for its definite integral in terms of the integral of the series,

$$\begin{aligned}\int_a^b e^{x^2}dx &= \int_a^b \sum_{n=0}^{\infty} \frac{x^{2n}}{n!}dx \\ &= \sum_{n=0}^{\infty} \frac{1}{n!}\int_a^b x^{2n}dx \\ &= \sum_{n=0}^{\infty} \frac{1}{n!}\left[\frac{x^{2n+1}}{2n+1}\right]_{x=a}^{b}, \\ &= \sum_{n=0}^{\infty} \frac{1}{n!}\frac{b^{2n+1}-a^{2n+1}}{2n+1}, \end{aligned} \tag{3}$$

where $[f(x)]_{x=a}^{b} = f(b) - f(a)$.

Note that this integration of the Taylor series is itself still an infinite sum. If we again only sum a *finite* number of the terms, then we will have an approximation of the definite integral. This approximation will become more and more accurate as we include more and more terms from the series.

You will need equation (3) for this assignment.

3.4 Riemann Sum

The area under the curve of a function can instead be calculated from a series of rectangles with a defined width and a height given by the function (Figure 1). This is known as the Riemann sum and, in the limit of zero-width (and infinitely many) rectangles, this summation approaches the definite integral,

$$\int_a^b f(x)dx = \lim_{\Delta x \to 0} \sum_{i=0}^{n-1} f(x_i)\Delta x$$

where

$$\begin{aligned} x_i &= a + i\,\Delta x, \quad \text{and} \\ n &= \frac{b-a}{\Delta x}. \end{aligned}$$

Note that this definition of the definite integral via the Riemann sum is yet another sum of an infinite series of terms in the limit of zero-width rectangles. If we sum only a *finite* number of very small (but nonzero-width) rectangles, then the Riemann sum will be an approximation of the definite integral. This approximation will also become more and more accurate as we include more and more terms from the series (i.e., as we consider smaller- and smaller-width rectangles).

The finite Riemann sum of the definite integral, $\int_a^b e^{x^2}dx$, using N terms is therefore given by,

$$\int_a^b e^{x^2}dx \approx \sum_{i=0}^{N-1} e^{x_i^2}\Delta x \tag{4}$$

 ©2026 Queen's University

where

$$\Delta x = \frac{b-a}{N}, \text{ and}$$
$$x_i = a + i\,\Delta x.$$

You will need equations (4) for this assignment. **Note that this summation is from 0 to $N-1$.**

3.5 Guidance and Suggestions

You may work on this assignment however you wish, but below is a guideline on how you may wish to approach this assignment.

In a single `apsc143assign2.c` file:

1. Create signatures for the required functions and an `int main(void)` function.
2. Write the details of the `inverseFactorial` function.
3. Call `inverseFactorial` in your `main` and print its output to confirm the results with Table 1b.
4. Repeat steps 2 & 3 separately for `taylorSeries`, `integralTaylorSeries`, and `riemannSum`.

Some additional hints that may help you complete the assignment:

- The relationship between the summation notation in math and a `for` loop in C can be defined as

$$\sum_{i=0}^{K-1} \ldots \iff \texttt{for (int i = 0; i < K; i++)} \{\ldots\}$$

- If when you compile your program you get the errors:

```
undefined reference to ‘fabs’
undefined reference to ‘exp’
undefined reference to ‘pow’
```

  then first make sure you have included `math.h` at the top of your `apsc143assign2.c` file and then add the following line to the end of your `CMakeLists.txt`:

```
target_link_libraries(EXECUTABLE_NAME m)
```

  where `EXECUTABLE_NAME` should be replaced by the name of your executable, which you will also see in the preceding line of the form: `add_executable(EXECUTABLE_NAME apsc143assign2.c)`.

 ©2026 Queen's University

Table 1: Expected Results

(a) Analytical result of e^{x^2}

x	e^{x^2}
-3.0	8103.0839
-1.5	9.4877
0.0	1.0000
0.5	1.2840
2.0	54.5982

(b) Inverse factorial, $\frac{1}{n!}$

n	inverseFactorial
0	1.0000
1	1.0000
3	0.1667
5	0.0083
10	0.0000

(c) Taylor series approximation of e^{x^2}

x	minTermSize	taylorSeries
-3.0	0.0001	8103.0839
-1.5	1.0000	8.7476
-1.5	0.1000	9.4083
-1.5	0.0100	9.4825
-1.5	0.0001	9.4877
0.0	0.0001	1.0000
0.5	0.0001	1.2840
2.0	0.0001	54.5981

(d) Analytic integral of the Taylor series approximation of e^{x^2}

lowerLimit	upperLimit	minTermSize	integralTaylorSeries
-2.0	0.0	1.0000	14.8362
-2.0	0.0	0.1000	16.3360
-2.0	0.0	0.0100	16.4396
-2.0	0.0	0.0001	16.4526
-0.5	1.5	0.0001	4.6080

(e) Riemann sum of e^{x^2}

lowerLimit	upperLimit	numRectangles	riemannSum
-2.0	0.0	3000	16.4705
-2.0	0.0	5000	16.4634
-2.0	0.0	10000	16.4580
-2.0	0.0	100000	16.4532
-0.5	1.5	100000	4.6080

 ©2026 Queen's University

4 Submission Instructions

Create your program and upload it to Gradescope for grading. Your program file **must be named** `apsc143assign2.c` in order for your assignment to be graded. Do not include any personal information (student number, name, etc.) in your submission.

Include a comment in your code attesting to the originality of your work and that you have not used Generative AI.

Refer to the assignment rubric below for a detailed breakdown of the grading criteria. Your submission must adhere to the assignment rules as outlined in the submission policy document for this course, which can also be found on OnQ.

There is zero tolerance for plagiarism in this course. The auto grading software will automatically flag potential cases of plagiarism, which will be reviewed by the teaching team.

More information on assignment submissions can be found on OnQ, and information on the specific definition and repercussions of plagiarism can be found in the "Begin Here (About This Course)" module.

4.1 Assignment Rubric

Test case marks are based purely on code output when given specific inputs. Code design, style, and commenting are graded by the teaching team looking at your code. Use comments effectively to not only get full marks in the comments section, but to also make your code clearer to the marking TA.

A portion of each assignment's marks will be given for your understanding of your code. If your code only uses the concepts taught so far in the course, then you will receive these marks automatically. If your code uses other concepts, including those not yet presented or not covered in the course, then in order to receive these marks you may be asked to explain to a member of the teaching team how your code works and why you chose to solve the problem that way.

 ©2026 Queen's University

Table 2: Marking Rubric — Output

Test Cases (Marks will only be awarded if the code compiles)	**Level 3 (3 points)**	**Level 2 (2 points)**	**Level 1 (1 points)**	**N/A (0 points)**	**Criterion Score**
Assignment Correctness - `inverseFactorial`	Actual output perfectly matches expected output.	Some of the required values are correct.	Few output values are correct.	The code did not compile or crashed during runtime.	/3
Assignment Correctness - `taylorSeries`	Actual output perfectly matches expected output.	Some of the required values are correct.	Few output values are correct.	The code did not compile or crashed during runtime.	/3
Assignment Correctness - `integralTaylorSeries`	Actual output perfectly matches expected output.	Some of the required values are correct.	Few output values are correct.	The code did not compile or crashed during runtime.	/3
Assignment Correctness - `riemannSum`	Actual output perfectly matches expected output.	Some of the required values are correct.	Few output values are correct.	The code did not compile or crashed during runtime.	/3
Assignment Correctness - Hidden Test Cases	Actual output perfectly matches expected output.	Some of the required values are correct.	Few values are correct.	The code did not compile or crashed during runtime.	/3

©2026 Queen's University

Table 3: Marking Rubric — Style

Code Style	**Level 2 (2 points)**	**Level 1 (1 points)**	**Level 0 (0 points)**	**Criterion Score**
Function Organization	Code is organized effectively into subfunctions to balance code reuse and complexity.	Code is organized into subfunctions but code reuse remains or functions result in unnecessary additional code complexity.	Code is not organized into subfunctions and significant code reuse exists.	/2
Coding Style	The coding style is consistent, clear, and well organized.	1 or more instances exist where coding style is not considered.	The coding style lacks clarity.	/2
Commenting	Code is well documented. Comments provide clarity for all tasks and logic.	Few comments are used, major tasks are undocumented and logic is difficult to follow. Comments included are vague and ineffective.	No meaningful comments are used.	/2

©2026 Queen's University

Table 4: Marking Rubric — Understanding

Code Understanding (Marks will only be awarded if the code compiles)	**Level 3 (9 points)**	**Level 2 (6 points)**	**Level 1 (3 points)**	**Level 0 (0 points)**	**Criterion Score**
Code Understanding	Student uses concepts covered in class that are assumed to be understood or uses concepts not covered in the class and can clearly explain and justify their submitted code to a member of the teaching team.	Student uses concepts not covered in class and can explain their submitted code to a member of the teaching team. Their justification of aspects of their submitted code to a member of the teaching team lacks clarity.	Student uses concepts not covered in class. Their explanation and justification of aspects of their submitted code to a member of the teaching team lacks clarity.	Student uses concepts not covered in class and cannot explain nor justify aspects of their submitted code to a member of the teaching team.	/9

©2026 Queen's University